Unterricht vs. Home Control

Authentifizierung, Autorisierung, Filter und Validierung

Zusammenfassung der Unterrichtsstunde vom 30.10.2026 (Blog-API) und Abgleich mit dem Stand von Home Control auf dem Branch `feature/authentication` (Commit 5a5dbad).

Kurzfazit

• **Die Grundlagen sind gleich:** JWT mit HS256, geheimer Schlüssel in den User Secrets, dieselben Prüfregeln (Signatur, Issuer, Audience, Ablauf), `MapInboundClaims = false`, derselbe Passwort-Hasher, 401 und 403 richtig eingesetzt.
• **Bei der Sicherheit ist Home Control weiter:** Refresh-Token mit Rotation und Reuse Detection, 15 statt 60 Minuten, Rollen im Token mit Admin-Regel, geschützte Registrierung, Schlüssel ohne Umweg über Webseite und Shell-Verlauf, automatische Tests statt nur Postman.
• **Beim .NET-10-Komfort ist der Unterricht moderner:** `AddValidation()` mit DataAnnotations, TypedResults mit `Results<…>`, OpenAPI-Angaben (`WithTags`, `WithSummary`, `ProducesProblem`) und die Hilfsmethode `GetUserId()`. Das kann Home Control übernehmen (Abschnitt 4).

Inhalt

1 Was im Unterricht dran war
2 Filter und Validierung (Lektion)
3 Vergleich Punkt für Punkt
4 Was Home Control übernehmen sollte
5 Wo Home Control weiter ist (für die Präsentation)
6 Code nebeneinander
7 Merksätze

1 Was im Unterricht dran war

Statuscodes (MDN)

Code	Bedeutung	Merke
400 Bad Request	Anfrage fehlerhaft (z. B. ungültiger Body)	Validierungsfehler
401 Unauthorized	Gemeint ist *unauthenticated*: Wer du bist, ist unbekannt	Name historisch irreführend
403 Forbidden	Identität bekannt, aber keine Rechte	Neu anmelden hilft nicht
404 Not Found	Ressource gibt es nicht	Darf statt 403 kommen, um zu verbergen, dass etwas existiert

JWT im Debugger (jwt.io)

- Ein Token aus der Blog-API enthält aud, `iss` (beide „BlogApi“), `sub` (User-Id), `email`, `name`, `iat`, `nbf`, exp.
- Der Inhalt ist ohne Schlüssel lesbar. Die Anzeige „Invalid Signature“ zeigt nur: Ohne den richtigen Schlüssel kann jwt.io die Signatur nicht bestätigen.
- exp minus `iat` = 3600 Sekunden, das Token gilt also **60 Minuten** (`Jwt:ExpiryMinutes`, Standard 60).

Geheimer Schlüssel in den User Secrets

```
dotnet user-secrets init --project BlogApi          # legt UserSecretsId in der .csproj an
openssl rand -hex 32                                # 32 Zufallsbytes als 64 Hex-Zeichen
dotnet user-secrets set "Jwt:Key" "<schluessel>"    # Schlüssel speichern (nicht im Repo)
```

Im Unterricht wurde zusätzlich random.org als Quelle für den Schlüssel gezeigt.

Program.cs

- `AddIdentityCore<User>` mit `RequireUniqueEmail`, `AddRoles<IdentityRole<Guid>>`, `AddEntityFrameworkStores`: ASP.NET Core Identity verwaltet Benutzer und Passwörter.
- `AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(…)` mit `MapInboundClaims = false`. Fehlt `Jwt:Key`, bricht der Start ab (`?? throw`).
- `TokenValidationParameters`: Signaturschlüssel, Issuer, Audience, Lebensdauer prüfen, `ClockSkew = TimeSpan.Zero` (keine Toleranz).
- `AddAuthorization()`, Services per `AddScoped<IAuthService, AuthService>`.

DTOs, AuthService und Endpunkte

- `LoginDto([Required, EmailAddress] string Email, [Required] string Password)`: Regeln stehen als Attribute am DTO.
- `RegisterAsync`: `UserManager.CreateAsync(user, password)`. Fehler aus `IdentityResult` werden gruppiert und als `ValidationProblem` (400) zurückgegeben.
- `LoginAsync`: `FindByEmailAsync` und `CheckPasswordAsync`. Stimmt etwas nicht: `null` → 401, ohne zu verraten, ob die E-Mail existiert.
- `CreateToken`: `SecurityTokenDescriptor` mit Claims `sub`, `email`, `name` und HMAC-SHA256.
- `MapGroup("/auth").WithTags("Auth")` mit `/register`, `/login` und `/me` (`RequireAuthorization()`). Rückgabetypen als `TypedResults` mit `Results<Ok<…>, UnauthorizedHttpResult>`.
- `ClaimsPrincipalExtensions.GetUserId()`: liest die User-Id aus dem Token, wiederverwendbar in allen Endpunkten.

Autorisierung an Beiträgen (Posts)

- Postman: `GET /auth/me` ohne Token → **401** mit ProblemDetails. Mit Bearer-Token → `POST /posts` → **201 Created**.
- Beim Anlegen kommt die User-Id **aus dem Token** (`CreateAsync(user.GetUserId(), dto)`), nicht mehr aus dem Body. Sonst könnte jeder im Namen anderer schreiben.
- Ändern und Löschen: `if (post.UserId != user.GetUserId()) return TypedResults.Forbid();` → **403**. Das ist **ressourcenbasierte Autorisierung**: Nur der Autor darf seinen Beitrag ändern.

2 Filter und Validierung (Lektion)

Problem: Minimal APIs nehmen jedes JSON an. Ein leeres `{}` erzeugt einen Eintrag mit `null`-Feldern, obwohl `string` nicht nullable ist. Nullable-Referenztypen prüft nur der Compiler, nicht der JSON-Deserialisierer.

Eingebaute Validierung in .NET 10

```
public record CreateEntryDto(
    [Required, StringLength(100, MinimumLength = 1)] string Title,
    [Required, StringLength(10_000, MinimumLength = 1)] string Content);

builder.Services.AddValidation();       // prüft Attribute vor dem Handler
builder.Services.AddProblemDetails();   // Fehler im ProblemDetails-Format
```

- Ungültige Anfrage → **400** mit `application/problem+json` und `errors` je Feld. Der Handler läuft dann gar nicht.
- Schlüssel in `errors` sind die C#-Namen in PascalCase (`Title`), der JSON-Body nutzt camelCase.
- Funktioniert auch für Routen- und Query-Parameter: `[Range(1, 50)] int take`.
- Häufige Attribute: `[Required]`, `[StringLength]`, `[MinLength]`/`[MaxLength]`, `[Range]`, `[EmailAddress]`, `[Url]`, `[RegularExpression]`.

> **Falle bei Werttypen:** `Guid`, `int` oder Enums sind nie `null`. Fehlt der Wert, steht dort `Guid.Empty` bzw. `0` und `[Required]` ist zufrieden. Lösung: nullable machen und `[Required]` setzen, also `[Required] Guid? UserId`.

Regeln, die Daten brauchen (z. B. „E-Mail schon vergeben“, „User existiert“), gehören in Service oder Handler. Mit `TypedResults.ValidationProblem(…)` kommt dasselbe Antwortformat heraus.

Endpunktfilter

Ein Filter implementiert `IEndpointFilter` und läuft um einen Handler herum. Er kann vorher prüfen und abbrechen (Kurzschluss) oder nach `await next(context)` das Ergebnis auswerten. Er wird per `AddEndpointFilter<T>()` an einen Endpunkt oder eine Gruppe gehängt.

- `RejectEmptyIdFilter`: lehnt `/users/00000000-0000-0000-0000-000000000000` mit 400 ab. Die Routenbeschränkung `{id:guid}` lässt die leere GUID nämlich durch.
- `TimingFilter`: misst die Dauer des Handlers und schreibt sie ins Log. Filter bekommen Services per Konstruktor (DI).

	Middleware	Endpunktfilter
Läuft für	jede Anfrage	nur zugeordnete Endpunkte/Gruppen
Zeitpunkt	vor dem Routing bzw. drumherum	nach dem Binden der Parameter, direkt vor dem Handler
Sieht Handler-Argumente	nein	ja (`context.Arguments`)
Gut für	Logging aller Anfragen, Fehler, CORS, Authentifizierung	Regeln für bestimmte Endpunkte

Vor .NET 10 hat man dafür eigene `ValidationFilter<T>` mit `Validator.TryValidateObject` geschrieben. Das ist mit `AddValidation()` nicht mehr nötig.

3 Vergleich Punkt für Punkt

Thema	Unterricht (Blog-API)	Home Control	Ergebnis
401 / 403	401 ohne Token, 403 bei fremdem Beitrag	401 ohne Token, 403 bei fehlender Admin-Rolle, mit Tests abgesichert	**gleich**
JWT-Prüfung	Signatur, Issuer, Audience, Lifetime, `MapInboundClaims=false`	dasselbe, aber zentral in `TokenService.CreateValidationParameters` für Middleware und Tests	**gleich**
Schlüssel lesen	zweimal: in Program.cs und im AuthService	einmal: `JwtOptions` als Singleton	**Home Control weiter**
Fehlender Schlüssel	Start bricht immer ab	Development: Ersatzschlüssel mit Warnung. Sonst: Start bricht ab	**anderer Ansatz**
Schlüssel erzeugen	`openssl rand` oder random.org, Wert steht im Befehl (Shell-Verlauf)	Skript erzeugt 64 Byte lokal und übergibt per stdin, Wert ist nie sichtbar	**Home Control weiter**
ClockSkew	0 Sekunden	30 Sekunden (Uhren weichen leicht ab)	**anderer Ansatz**
Token-Laufzeit	60 Minuten	15 Minuten	**Home Control weiter**
Refresh-Token	noch nicht gezeigt	Zufallswert, nur Hash in der DB, HTTP-only Cookie, Rotation, Reuse Detection	**Home Control weiter**
Claims	`sub`, `email`, `name`	`sub`, `email`, `role`	**anderer Ansatz**
Benutzer	ASP.NET Core Identity (`UserManager`, Tabelle AspNetUsers)	eigene Tabelle Users, nur `PasswordHasher<User>` aus Identity	**anderer Ansatz**
Passwort-Hash	PBKDF2 über Identity	PBKDF2 über denselben `PasswordHasher`	**gleich**
Passwortregeln	Identity-Standard: Groß/klein, Ziffer, Sonderzeichen, mind. 6	mind. 8, max. 128 Zeichen	**anderer Ansatz**

Thema	Unterricht (Blog-API)	Home Control	Ergebnis
Registrierung	offen für alle	erster Benutzer wird Admin, danach nur Admin	**Home Control weiter**
Login-Fehler	gleiche Antwort für E-Mail unbekannt und Passwort falsch	gleich	**gleich**
Rollen / Rechte	Rollen vorbereitet, Rechte über Besitz (Autor)	Policy `AdminOnly` (RBAC) auf schreibenden Endpunkten	**anderer Ansatz**
User-Id aus Token	`GetUserId()` als Erweiterungsmethode	einmal direkt in `GetProfile`	**Unterricht moderner**
Eingaben prüfen	`AddValidation()` + DataAnnotations am DTO	eigene Prüfmethoden im Endpunkt	**Unterricht moderner**
Rückgabetypen	`TypedResults` + `Results<…>`	`IResult` + `Results.…`	**Unterricht moderner**
OpenAPI-Doku	`WithTags`, `WithName`, `WithSummary`, `Produces…`	keine	**Unterricht moderner**
Endpunktfilter	`RejectEmptyIdFilter`, `TimingFilter`	keine (bisher nicht nötig)	**Unterricht moderner**
Interfaces	`IAuthService`, `IPostService` …	konkrete Klassen, Interface nur bei Blink	**anderer Ansatz**
Testen	von Hand mit Postman	xUnit: Token-, Auth- und Endpunkt-Tests (WebApplicationFactory)	**Home Control weiter**
Frontend	–	Login-Seite, Bearer automatisch, Refresh mit Sperre	**Home Control weiter**

4 Was Home Control übernehmen sollte

Sortiert nach Nutzen. Keiner der Punkte ist eine Sicherheitslücke. Es geht um weniger Code, bessere API-Doku in Scalar und den aktuellen .NET-10-Stil.

#	Übernehmen	Warum	Aufwand
1	`AddValidation()` + DataAnnotations an den Request-DTOs (Auth, Räume, Geräte)	Regeln stehen am DTO statt verteilt im Endpunkt. Der Handler läuft nur mit gültigen Daten. Weniger eigener Code.	mittel
2	`TypedResults` und `Results<…>` als Rückgabetyp	Der Compiler prüft, ob nur erlaubte Antworten zurückkommen. Scalar zeigt die Antworten automatisch.	mittel
3	`WithTags`, `WithSummary`, `ProducesProblem(401/403)`	Bessere API-Doku in Scalar, gut für die Präsentation.	klein
4	`ClaimsPrincipalExtensions.GetUserId()`	Wird gebraucht, sobald mehr Endpunkte wissen müssen, wer anfragt.	klein
5	Endpunktfilter, z. B. `RejectEmptyIdFilter`	Leere GUID gibt dann 400 statt 404. Nur Feinschliff.	klein

Bewusst nicht übernehmen: ASP.NET Core Identity komplett. Die eigene, kleine Lösung ist für die Präsentation leichter zu erklären. Gehasht wird mit demselben `PasswordHasher`. Eine Sperre nach Fehlversuchen fehlt **beiden** Projekten: `UserManager.CheckPasswordAsync` zählt keine Fehlversuche, das würde erst `SignInManager.CheckPasswordSignInAsync` mit Lockout tun. Für Home Control wäre später Rate Limiting auf `/api/auth/login` der passende Schritt.

Achtung beim Umstieg auf AddValidation: Die Fehlerschlüssel heißen dann `Email` statt `email`. Das Frontend zeigt nur die erste Meldung an, ist also nicht betroffen. Beim Login keine Passwortlänge prüfen (nur `[Required]`), sonst verrät die Antwort die Passwortregeln.

5 Wo Home Control weiter ist (für die Präsentation)

Punkt	Was du sagen kannst
Refresh-Token mit Rotation	Das Access Token lebt nur 15 Minuten. Damit niemand sich ständig neu anmelden muss, gibt es ein Refresh-Token. Das gilt nur einmal und liegt im HTTP-only Cookie. In der Datenbank steht nur sein Hash.
Reuse Detection	Taucht ein schon benutztes Refresh-Token wieder auf, wurde es wahrscheinlich gestohlen. Dann werden alle Sitzungen des Benutzers beendet.
Geschützte Registrierung	In einem Smart Home soll sich nicht jeder ein Konto anlegen können. Der erste Benutzer wird Admin, danach legt nur ein Admin Konten an.
Schlüssel sicher erzeugen	Der Schlüssel wird lokal erzeugt und per stdin übergeben. Er steht weder im Shell-Verlauf noch hat ihn eine fremde Webseite (wie bei random.org) je gesehen.
Eine Stelle für die JWT-Regeln	Middleware und Tests nutzen dieselben Prüfregeln. Ändert man eine Regel, gilt sie überall.
Automatische Tests	Statt nur von Hand in Postman: Tests für Token, Login, Rotation, Ablauf, Widerruf, 401 und 403. Sie laufen bei jedem `dotnet test`.
Frontend fertig	Login-Seite, Token geht automatisch mit, bei 401 genau ein Refresh mit Sperre (wegen React StrictMode und Reuse Detection).

Bewusst anders, nicht besser oder schlechter

- **ClockSkew 0 gegen 30 Sekunden:** 0 ist am strengsten. 30 Sekunden verzeihen kleine Uhrabweichungen zwischen Geräten (z. B. iPad und Server). Der Standard wäre 5 Minuten, das wären beide zu lasch.
- **Besitz gegen Rolle:** Ein Blog hat Beiträge mit Autor, deshalb Besitz-Prüfung. In Home Control gehören Räume und Geräte dem ganzen Haushalt, deshalb Rollen (Admin darf ändern, alle dürfen lesen).
- **Fehlender Schlüssel:** Der Unterricht bricht immer ab. Das ist einfach und sicher. Home Control startet in Development mit Ersatzschlüssel, damit Tuya und Blink weiterlaufen. In Production bricht es ebenfalls ab.
- **Gelöschter Benutzer mit gültigem Token:** Der Unterricht antwortet bei `/me` mit 404. Home Control antwortet bei `/profile` mit 401, weil das Token zu keinem Benutzer mehr gehört.

6 Code nebeneinander

Rechte prüfen: Besitz (Unterricht) gegen Rolle (Home Control)

```
// Unterricht: nur der Autor darf ändern (ressourcenbasiert)
if (post.UserId != user.GetUserId())
    return TypedResults.Forbid();                          // 403

// Home Control: nur Admin darf ändern (rollenbasiert, als Regel am Endpunkt)
rooms.MapPut("/{id:guid}", UpdateRoom)
     .RequireAuthorization(AuthPolicies.AdminOnly);      // 401 ohne Token, 403 ohne Admin
```

Eingaben prüfen: DataAnnotations (Unterricht) gegen eigene Prüfung (Home Control)

```
// Unterricht: Regeln am DTO, AddValidation() prüft vor dem Handler
public record LoginDto([Required, EmailAddress] string Email, [Required] string Password);

// Home Control: Prüfung im Endpunkt
var errors = ValidateEmailAndPassword(request.Email, request.Password, checkPasswordRules: false);
if (errors.Count > 0) return Results.ValidationProblem(errors);
```

Token-Laufzeit und Toleranz

```
// Unterricht                                    // Home Control (appsettings.json + TokenService)
GetValue("Jwt:ExpiryMinutes", 60)                "AccessTokenMinutes": 15,  "RefreshTokenDays": 30
ClockSkew = TimeSpan.Zero                        ClockSkew = TimeSpan.FromSeconds(30)
```

7 Merksätze

- 401 = nicht angemeldet (eigentlich „unauthenticated“). 403 = angemeldet, aber nicht erlaubt.
- Wer anfragt, steht im Token (sub), nie im Body. Sonst gibt sich jeder als jemand anderes aus.
- Ein JWT ist lesbar. „Invalid Signature“ heißt nur: Der Prüfer hat den Schlüssel nicht.
- Schlüssel lokal erzeugen (`openssl rand`, Skript), nicht von Webseiten, und nicht als Befehl in den Shell-Verlauf.
- `AddValidation()` + DataAnnotations: ungültige Anfrage → 400, Handler läuft gar nicht.
- Pflicht-Werttypen nullable machen: `[Required] Guid?`.
- Regeln mit Datenbankzugriff gehören in den Service, Antwort per `ValidationProblem`.
- Middleware gilt für alle Anfragen, Endpunktfilter nur für bestimmte Endpunkte und sehen die Argumente.
- Rollenbasiert (RBAC) prüft die Rolle, ressourcenbasiert prüft den Besitz. Beides führt bei Verbot zu 403.